FedPEFT-IMU: Federated Parameter-Efficient Adaptation of Pretrained IMU Encoders for Human Activity Recognition under Device and Data Heterogeneity

First Author, *Student Member, IEEE*, Second Author, *Member, IEEE*

***Abstract*—Abstract to be written once the results section is final.**

***Index Terms*—Federated learning, human activity recognition, inertial sensors, low-rank adaptation, parameter-efficient fine-tuning, foundation models, personalization, heterogeneity.**

I. Introduction

[Introduction: to be written.]

II. Related Work

[Related work: to be written.]

III. Proposed Method: FedPEFT-IMU

This section formalizes the learning problem (Section III-A), presents the architecture and the role assigned to every parameter (Sections III-B and III-C), derives the client objective (Section III-D) and the server aggregation rules together with their exactness guarantees (Sections III-E and III-F), explains how label skew is handled (Section III-G), and describes inference for enrolled and new users (Section III-H). The complete procedure is summarized in Algorithm 1, and its cost is analyzed in Section III-J. Table I lists the main notation.

A. Problem Setting and Design Goals

We consider K users, each holding one wrist-worn inertial measurement unit (IMU) that records a continuous stream of tri-axial acceleration. The stream of user k is segmented into non-overlapping windows $\mathbf{x} \in \mathbb{R}^{3\times L}$ (here 10 s at 30 Hz, $L = 300$) with activity labels $y \in \{1, \dots, C\}$, giving a chronologically ordered dataset $\mathcal{D}_k = \{(\mathbf{x}_{k,i}, y_{k,i})\}_{i=1}^{|\mathcal{D}_k|}$. Raw windows never leave the device. A server orchestrates training over rounds $t = 0, \dots, T-1$ in which a random subset $\mathcal{S}_t$ of $\lceil \rho K \rfloor$ clients participates.

The setting combines four kinds of heterogeneity that, together, distinguish federated human activity recognition (HAR) from the image and text benchmarks on which most federated parameter-efficient fine-tuning (PEFT) methods were developed:

(H1) *Label skew*: $\boldsymbol{\pi}_k \neq \boldsymbol{\pi}$; rare activities (e.g., bicycling) are absent from most clients (Section IV-D).

Manuscript draft. Code, user partitions and all generated statistics are released with the paper.

TABLE I
Main notation.

Symbol	Meaning
K, $\mathcal{S}_t$, ρ	number of clients, clients selected in round t, participation rate
$\mathcal{D}_k^{\mathrm{tr}}$, $\mathcal{D}_k^{\mathrm{te}}$	chronological training / test part of client k
n_k, $n_{k,c}$	training windows of client k (in total / of class c)
C, $\boldsymbol{\pi}_k$, $\boldsymbol{\pi}$	classes, label prior of client k, global label prior
Π_E, a_k	fixed preprocessor of encoder E, local input adapter
$\bar{W}_\ell$, $\mathcal{L}$	frozen base weight of adapted layer ℓ, set of adapted layers
B_ℓ, A_ℓ, r_k, s	LoRA factors, LoRA rank of client k, scaling $s = \alpha/r$
ψ_k	local normalization parameters and statistics
g_θ, h_k	shared projection head, personal classifier
$\mathbf{z}$, $\mathbf{e}$, $\bar{\mathbf{e}}$	encoder feature, embedding ($D = 64$), $\mathbf{e}/\|\mathbf{e}\|_2$
$\mathbf{p}_{k,c}$, $\mathbf{P}_c$	local and global prototype of class c
$\tilde{p}_k$	aggregation weight $n_k / \sum_{j\in\mathcal{S}_t} n_j$
$B_{[r]}$, $A_{[r]}$	first r columns of B / first r rows of A
$\Delta_{k,\ell}$	change of client k's LoRA product in one round
N_k, Γ	$C \times C$ transition counts of client k, HMM transition matrix

(H2) *Feature shift*: $p_k(\mathbf{x} \mid y)$ differs across users because of body size, gait, device orientation and wear habits.

(H3) *System heterogeneity*: devices differ in memory and compute; we model this by a per-device budget expressed as a LoRA rank $r_k \in \mathcal{R} = \{2, 4, 8\}$.

(H4) *Temporal dependence*: labels form long bouts, so consecutive windows are strongly correlated (Section IV-D).

A pretrained IMU encoder E with weights Ω_0, obtained from data outside the federation, is available to every participant. The design goals are (G1) high macro-F1 for *enrolled* users on their *future* data; (G2) useful predictions for *new* users with zero or a few minutes of labels; (G3) low uplink traffic; (G4) training within each device's budget r_k; and (G5) fairness, i.e., a high F1 for the worst-served users.

We cast the problem as personalized federated learning with shared parameters Θ and client-specific parameters ϕ_k:

$$\min_{\Theta} \sum_{k=1}^{K} p_k \min_{\phi_k} F_k(\Theta, \phi_k), \qquad p_k = \frac{n_k}{\sum_j n_j}, \tag{1}$$

where F_k is the local objective defined in Section III-D and $n_k = |\mathcal{D}_k^{\mathrm{tr}}|$.

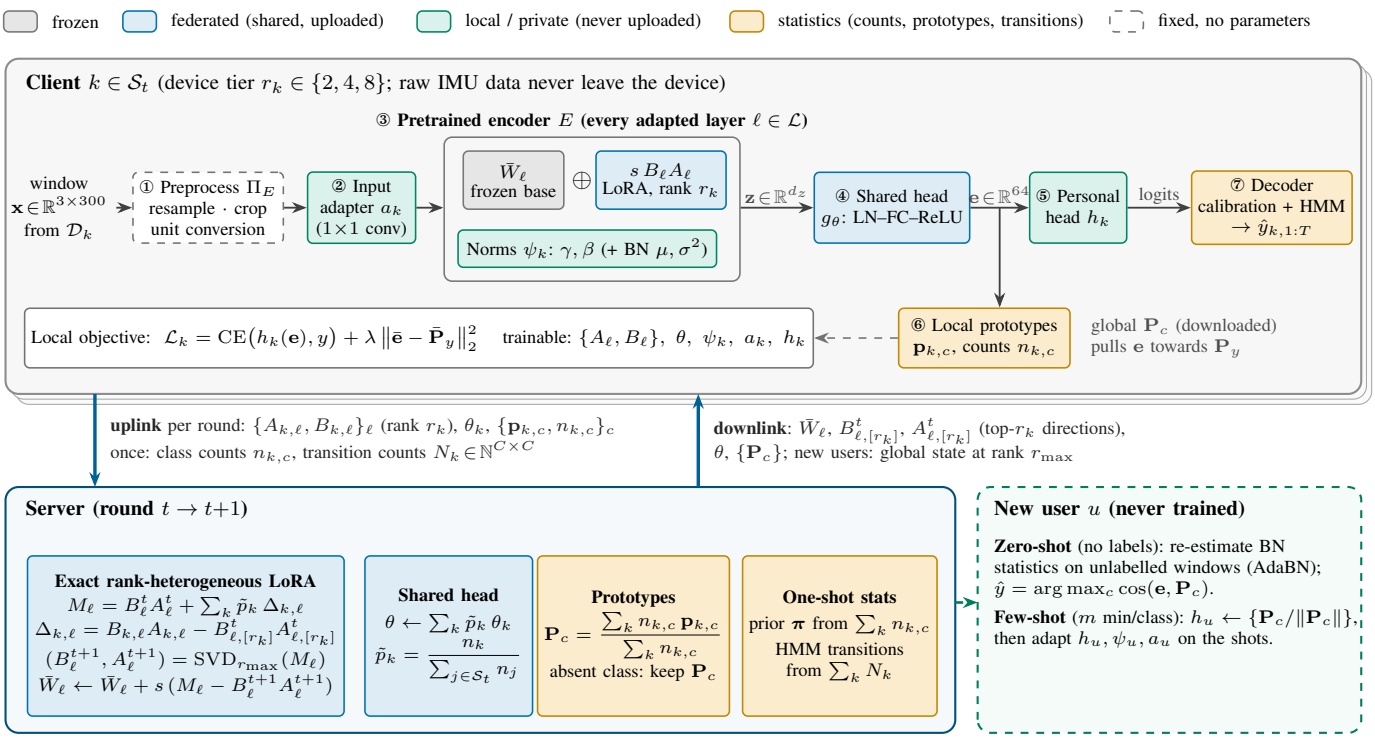


Fig. 1. FedPEFT-IMU architecture. Each client runs the same seven-component model ①–⑦ on top of a frozen pretrained IMU encoder. Parameters are partitioned by role: the base weights are frozen (grey); the low-rank adapters and the projection head are federated (blue) and trained at a device-specific rank r_k; the input adapter, the normalization layers and the classifier stay on the device (green); class prototypes and one-shot label statistics are the only additional information that is shared (amber). The server aggregates LoRA updates exactly in delta form, re-factorizes them by a truncated SVD and folds the remainder into the frozen weights (Fig. 2). New users are served without any training through the global prototypes.

B. Architecture Overview and Parameter Roles

Figure 1 shows the complete system. Its central design decision is to give every tensor of the client model exactly one of four *roles*, chosen according to what the tensor is expected to capture (Table II):

- *Frozen*: the pretrained weights Ω_0. They encode generic motion features learned from large external corpora and are never transmitted after deployment.
- *Federated* (Θ): the LoRA factors $\{A_\ell, B_\ell\}_{\ell\in\mathcal{L}}$ and the projection head g_θ. They adapt the representation to the target population and benefit from all users' data.
- *Local* ($\phi_k = \{a_k, \psi_k, h_k\}$): the input adapter, the normalization layers and the classifier. Following the evidence that feature shift is largely captured by normalization statistics [1] and label skew by the classifier layer [2], [3], these parts absorb (H1)–(H2) and are never uploaded, which also reduces what a server can infer about an individual.
- *Statistics*: per-class prototypes $\mathbf{p}_{k,c}$ and counts $n_{k,c}$, sent every round, and transition counts N_k, sent once. They are small ($C \times D$, C and C^2 numbers) and align the users' embedding spaces (Section III-F).

Because each method studied in this paper is defined solely by which roles are trained, uploaded or kept local (Table VIII), all methods share one implementation, and differences in the results can be attributed to the role assignment rather than to engineering details.

TABLE II
COMPONENTS OF THE FEDPEFT-IMU CLIENT MODEL AND THEIR ROLES.

#	Component	Role	Purpose
①	Preprocessor Π_E	fixed	match the encoder's rate, length and units
②	Input adapter a_k	local	device axes / channel mapping
③	Base weights $\bar{W}_\ell$	frozen	generic motion representation
	LoRA $B_\ell A_\ell$ (rank r_k)	federated	population-level adaptation
	Norms ψ_k	local	per-user feature statistics (H2)
④	Shared head g_θ	federated	common 64-d embedding space
⑤	Personal head h_k	local	per-user decision rule (H1)
⑥	Prototypes $\mathbf{p}_{k,c}$, $n_{k,c}$	statistics	cross-user alignment, zero-shot
⑦	Decoder (prior, HMM)	statistics	calibration, temporal context (H4)

C. Client Model

1) Encoder-agnostic preprocessing ①: Windows are stored once in a canonical format (3×300, 30 Hz, units of g). For each encoder E, a fixed operator Π_E maps them to the format used during pretraining: anti-aliased linear resampling to the encoder rate f_E, a centered crop to the encoder length L_E (e.g., 6 s for LIMU-BERT), and conversion to $\mathrm{m\,s^{-2}}$ where required. Π_E has no parameters and is identical on all devices.

2) Input adapter ②: A 1×1 convolution $a_k : \mathbb{R}^{3\times L_E} \rightarrow \mathbb{R}^{c_E \times L_E}$ maps the device channels to the c_E input channels of the encoder. It is initialized to the identity on the accelerometer channels and to zero elsewhere (e.g., on the gyroscope channels expected by LIMU-BERT), so training starts exactly from the pretrained model. With $3c_E + c_E$ parameters, it can learn

device-specific axis permutations, sign flips and gains, and it is therefore kept local.

3) Frozen encoder with low-rank adapters ③: For every adapted layer $\ell \in \mathcal{L}$ with frozen weight $\bar{W}_\ell \in \mathbb{R}^{d_\ell^{\text{out}} \times d_\ell^{\text{in}}}$, LoRA [4] learns $B_\ell \in \mathbb{R}^{d_\ell^{\text{out}} \times r}$ and $A_\ell \in \mathbb{R}^{r \times d_\ell^{\text{in}}}$ and uses the effective weight

$$W_\ell = \bar{W}_\ell + s\, B_\ell A_\ell, \qquad s = \alpha / r. \tag{2}$$

B_ℓ is initialized to zero and A_ℓ with Kaiming-uniform values, so $W_\ell = \bar{W}_\ell$ at the start. We set $\alpha = 2r$, which gives the same scaling $s = 2$ for every rank; products trained at different ranks are therefore directly comparable, a prerequisite for aggregating them. IMU encoders are largely convolutional, so LoRA is defined for convolutions as well: a kernel $\mathcal{K} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_1 \times \cdots}$ is treated as the matrix $\mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} k_1 \cdots)}$. The low-rank branch is computed as a convolution with r output channels and the original kernel geometry (stride, dilation and padding mode), followed by a $1{\times}1$ convolution back to C_{out} channels. All factors are stored as two-dimensional matrices irrespective of the layer type, which makes the server-side aggregation independent of the encoder architecture. Without LoRA dropout, the forward pass uses the merged kernel of Eq. (2); this is mathematically identical, autograd still only updates A_ℓ and B_ℓ, and it avoids two extra convolutions per layer.

4) Local normalization ③: The affine parameters (γ, β) of every LayerNorm and BatchNorm layer, and the running statistics (μ, σ^2) of BatchNorm layers, form ψ_k. They are trained and kept on the device, in the spirit of FedBN [1]. When the normalization parameters are not trained (methods V1–V4 in Table VIII), BatchNorm layers are kept in inference mode so that their pretrained statistics do not drift.

5) Shared projection head ④: $g_\theta(\mathbf{z}) = \mathrm{ReLU}\big(W_g\,\mathrm{Dropout}(\mathrm{LN}(\mathbf{z})) + \mathbf{b}_g\big)$ maps the encoder feature $\mathbf{z} \in \mathbb{R}^{d_z}$ to an embedding $\mathbf{e} \in \mathbb{R}^D$ with $D = 64$. The leading LayerNorm puts the very different feature scales of the four encoders ($d_z \in \{72, 256, 512, 1024\}$) on a common footing. Because g_θ is federated, all users embed activities into the same space, which is what makes prototypes comparable across users.

6) Personal classifier ⑤: $h_k(\mathbf{e}) = W_k \mathbf{e} + \mathbf{b}_k$ with $W_k \in \mathbb{R}^{C \times D}$ is kept local. A personal classifier may legitimately encode the user's own label distribution (a person who never cycles should rarely predict cycling), which a shared classifier cannot do.

7) Class prototypes ⑥: After local training, client k computes the mean embedding of each class it observes,

$$\mathbf{p}_{k,c} = \frac{1}{|\mathcal{I}_{k,c}|} \sum_{i \in \mathcal{I}_{k,c}} \mathbf{e}_{k,i}, \tag{3}$$

on a random subset $\mathcal{I}_{k,c}$ of at most 512 of its training windows, and reports $\mathbf{p}_{k,c}$ together with its true class count $n_{k,c}$.

The full forward pass of client k is

$$\begin{aligned} \mathbf{e} &= g_\theta\Big(E\big(a_k(\Pi_E(\mathbf{x}));\ \{\bar{W}_\ell + s\, B_\ell A_\ell\}_{\ell \in \mathcal{L}},\ \psi_k\big)\Big), \\ \hat{\mathbf{y}} &= h_k(\mathbf{e}). \end{aligned} \tag{4}$$

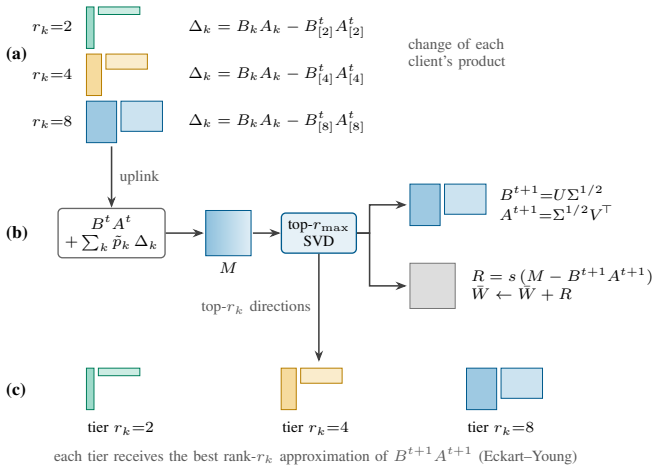


Fig. 2. Exact rank-heterogeneous aggregation for one adapted layer (glyph widths are proportional to the rank). (a) Each client uploads factors of its own rank r_k; the server forms the change Δ_k of the client's product relative to what the client received. (b) The weighted mean change is added to the current global product; the result M is re-factorized by a truncated SVD at rank $r_{\max}$, and the remainder R is folded into the frozen weight, so the effective model equals the aggregate of Proposition 1. (c) A device of rank r_k receives the leading r_k singular directions.

D. Local Objective

Given the global prototypes $\{\mathbf{P}_c\}_{c \in \mathcal{C}_P}$ of the previous round ($\mathcal{C}_P$ is the set of classes that already have one), client k minimizes

$$\begin{aligned} F_k(\Theta, \phi_k) = \frac{1}{n_k} \sum_{(\mathbf{x},y) \in \mathcal{D}_k^{\text{tr}}} \Big[&\mathrm{CE}\big(h_k(\mathbf{e}), y\big) \\ &+ \lambda\, \mathbf{1}[y \in \mathcal{C}_P]\, \big\|\bar{\mathbf{e}} - \bar{\mathbf{P}}_y\big\|_2^2 \Big], \end{aligned} \tag{5}$$

over the trainable set $\{A_\ell, B_\ell\}_{\ell \in \mathcal{L}} \cup \{\theta\} \cup \phi_k$, where bars denote ℓ_2-normalized vectors and $\lambda = 0.5$. The prototype term pulls each embedding towards the population-level center of its class and thereby counteracts the drift of the shared representation towards the local label distribution [5]. We compute it on the unit sphere, where

$$\big\|\bar{\mathbf{u}} - \bar{\mathbf{v}}\big\|_2^2 = 2 - 2\cos(\mathbf{u}, \mathbf{v}) \in [0, 4]. \tag{6}$$

Unlike the unnormalized distance, Eq. (6) cannot be decreased by shrinking all embeddings, and its magnitude does not depend on the feature scale of the encoder, so a single λ serves all four encoders. The classifier is trained with plain cross-entropy: h_k is personal, so the Bayes-optimal rule *for client* k includes its own prior $\boldsymbol{\pi}_k$ and no re-balancing is needed (Section III-G discusses the shared-classifier case).

E. Exact Rank-Heterogeneous LoRA Aggregation

1) Why naive averaging fails: Averaging the factors separately (FedAvg on A and B) does not average the models, because the product of averages differs from the average of products.

Lemma 1 (Bias of factor averaging): Let $\bar{B} = \sum_k \tilde{p}_k B_k$ and $\bar{A} = \sum_k \tilde{p}_k A_k$ with $\sum_k \tilde{p}_k = 1$. Then

$$\bar{B}\bar{A} - \sum_k \tilde{p}_k B_k A_k = -\sum_k \tilde{p}_k\, (B_k - \bar{B})(A_k - \bar{A}). \tag{7}$$

Proof: Expanding the right-hand side gives $-\sum_k \tilde{p}_k B_k A_k + \bar{B}\bar{A} + \bar{B}\bar{A} - \bar{B}\bar{A}$. ∎

The error is a weighted cross-covariance of the client factors. It vanishes only if the clients' factors do not vary, and it grows precisely when the clients' data differ, i.e., under (H1)–(H2). FedEx-LoRA [6] removes it for equal ranks by adding the residual $s(\sum_k \tilde{p}_k B_k A_k - \bar{B}\bar{A})$ to the frozen weight. With heterogeneous ranks, however, two further problems appear: the factors have different shapes, and a low-rank device cannot represent, and therefore cannot update, the trailing directions of the global adapter.

2) Delta-form aggregation: The server keeps, for every $\ell \in \mathcal{L}$, the base $\bar{W}^t_\ell$ (initially Ω_0) and global factors (B^t_ℓ, A^t_ℓ) at rank $r_{\max} = \max \mathcal{R}$. We drop the layer index when no confusion arises. Client k receives the truncated factors $(B^t_{[r_k]}, A^t_{[r_k]})$, trains them, and uploads (B_k, A_k) of rank r_k. The server computes the change of each client's product relative to what that client received,

$$\Delta_k = B_k A_k - B^t_{[r_k]} A^t_{[r_k]}, \tag{8}$$

adds the weighted mean change to the current global product, re-factorizes the result at rank $r_{\max}$ and folds the remainder into the frozen weight:

$$M = B^t A^t + \sum_{k \in \mathcal{S}_t} \tilde{p}_k \, \Delta_k, \tag{9}$$

$$U \Sigma V^\top = \mathrm{SVD}_{r_{\max}}(M),$$

$$(B^{t+1}, A^{t+1}) = \big(U\Sigma^{1/2},\ \Sigma^{1/2} V^\top\big), \tag{10}$$

$$\bar{W}^{t+1} = \bar{W}^t + s\,\big(M - B^{t+1} A^{t+1}\big). \tag{11}$$

Here $\tilde{p}_k = n_k / \sum_{j \in \mathcal{S}_t} n_j$ and $\mathrm{SVD}_{r_{\max}}$ returns the leading $r_{\max}$ singular triplets. Directions whose singular value is numerically zero ($\sigma_j < 10^{-6}\sigma_1$) would leave both factors at zero and block their gradients; for these we re-initialize the rows of A^{t+1} randomly and keep the columns of B^{t+1} at zero, which leaves the product unchanged. For large matrices ($\min(d^{\mathrm{out}}, d^{\mathrm{in}}) > 4r_{\max}$) a randomized SVD [7] with 8 oversampling vectors and 4 power iterations is used; as shown next, this does not affect exactness, because whatever the factors miss is kept in the residual.

Proposition 1 (Exactness): Let $W^t = \bar{W}^t + sB^tA^t$ be the effective global weight, and let $W^{\mathrm{start}}_k = \bar{W}^t + s\,B^t_{[r_k]}A^t_{[r_k]}$ and $W^{\mathrm{end}}_k = \bar{W}^t + sB_kA_k$ be client k's effective weight before and after local training. For any ranks $\{r_k\}$ and any factorization returned in Eq. (10),

$$W^{t+1} = W^t + \sum_{k \in \mathcal{S}_t} \tilde{p}_k \big(W^{\mathrm{end}}_k - W^{\mathrm{start}}_k\big). \tag{12}$$

Proof: By Eq. (11), $W^{t+1} = \bar{W}^{t+1} + sB^{t+1}A^{t+1} = \bar{W}^t + sM$. Substituting Eq. (9) gives $W^{t+1} = \bar{W}^t + sB^tA^t + \sum_k \tilde{p}_k\, s\Delta_k = W^t + \sum_k \tilde{p}_k\, s\Delta_k$, and $s\Delta_k = W^{\mathrm{end}}_k - W^{\mathrm{start}}_k$ by Eq. (8). ∎

Corollary 1 (Reduction to FedAvg): If all clients use $r_k = r_{\max}$, then $W^{\mathrm{start}}_k = W^t$ for every k, and Eq. (12) becomes $W^{t+1} = \sum_k \tilde{p}_k W^{\mathrm{end}}_k$: the effective models are averaged exactly as FedAvg [8] would average fully fine-tuned layers, and the rule coincides with FedEx-LoRA [6].

In other words, the server performs FedAvg on the *effective-weight updates* of the clients, regardless of their ranks and of how the update is factorized. Over rounds, the accumulated residual lets the global update grow beyond rank $r_{\max}$, while every device trains only $r_k(d^{\mathrm{in}} + d^{\mathrm{out}})$ parameters per layer.

3) Why the delta form matters: A natural alternative, used in spirit by product-space methods such as FlexLoRA [9], averages the zero-padded products, $M' = \sum_k \tilde{p}_k B_k A_k$. The following property shows that this alternative is biased under rank heterogeneity, whereas the delta form is not.

Proposition 2 (Consistency): Suppose that, after an aggregation, $B^tA^t = \sum_{j=1}^{r_{\max}} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top$ is in the balanced SVD form of Eq. (10), and that no client changes its factors ($B_k = B^t_{[r_k]}$, $A_k = A^t_{[r_k]}$). Then the delta-form rule returns $W^{t+1} = W^t$, whereas product averaging returns

$$M' = \sum_{j=1}^{r_{\max}} \varrho_j \, \sigma_j \mathbf{u}_j \mathbf{v}_j^\top, \qquad \varrho_j = \sum_{k \in \mathcal{S}_t:\ r_k \geq j} \tilde{p}_k \ \leq 1. \tag{13}$$

Proof: With unchanged factors, $\Delta_k = 0$ for all k, so $M = B^tA^t$ and $W^{t+1} = W^t$ by Proposition 1. For product averaging, the balanced form gives $B^t_{[r]}A^t_{[r]} = \sum_{j \leq r} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top$; summing over clients with weights $\tilde{p}_k$ and exchanging the sums yields Eq. (13). ∎

Under product averaging, the j-th direction of the global adapter is therefore multiplied by ϱ_j in every round, independently of the data. With tiers drawn uniformly from $\{2, 4, 8\}$, $\mathbb{E}[\varrho_j] = 1,\ \frac{2}{3},\ \frac{1}{3}$ for $j \in \{1, 2\}$, $\{3, 4\}$ and $\{5, \ldots, 8\}$: what the high-tier devices learn is diluted geometrically by the low-tier devices that never hold those directions. In the delta form, a client that does not hold a direction contributes a zero *change* to it rather than a zero *value*. Finally, because (B^t, A^t) is in balanced SVD form, the truncation sent to a rank-r_k device is the best rank-r_k approximation of the global adapter in the Frobenius and spectral norms (Eckart–Young–Mirsky [10]): weak devices receive the most informative part of the global update.

F. Aggregation of the Head, Prototypes and Statistics

The projection head is averaged as $\theta^{t+1} = \sum_{k \in \mathcal{S}_t} \tilde{p}_k \theta_k$. Prototypes are aggregated with the class counts as weights,

$$\mathbf{P}_c = \frac{\sum_{k \in \mathcal{S}_t} n_{k,c}\, \mathbf{p}_{k,c}}{\sum_{k \in \mathcal{S}_t} n_{k,c}}, \tag{14}$$

so a client contributes to a class only if it observes that class; a class that no participant observes in a round keeps its previous prototype. In addition, every client uploads two statistics once, before training: its class counts $\{n_{k,c}\}_c$ and its activity-transition counts $N_k \in \mathbb{N}^{C \times C}$, computed from its chronologically ordered training labels. The server forms the global prior $\pi_c \propto \sum_k n_{k,c}$ and the transition matrix of the temporal decoder (Section III-H). Together these statistics cost $C^2 + C = 42$ integers per client.

G. Learning under Label Skew

1) Why rare classes vanish with a shared classifier: With a shared classifier f and plain cross-entropy, the gradient with respect to the logit of class c is $q_c - \mathbf{1}[y = c]$, where $q =$

$\mathrm{softmax}(f(\mathbf{x}))$. On a client that never observes c, every sample therefore lowers the logit of c. In Capture-24, $74\,\%$ of the clients never cycle (Section IV-D); averaging their updates erases the class from the global model. In a preliminary configuration without the remedies described below, 35 of the 168 per-class F1 scores of the federated models (4 encoders $\times$ 7 methods $\times$ 6 classes) were zero: 17 for bicycling, 13 for vehicle and 5 for walking.

2) FedPEFT-IMU: The proposed design avoids this structurally: the classifier is personal (⑤), so no client's decision rule is pulled towards other clients' priors. The only shared class-specific quantities are the prototypes, and Eq. (14) averages each prototype only over the clients that have the class.

3) Shared-classifier methods: So that the comparison with methods that share the classifier (V0–V4 in Table VIII) is not dominated by the class-imbalance problem, those methods are equipped with three standard remedies. First, every client trains with a logit-adjusted (balanced-softmax) loss [11]–[13] that uses its *own* prior,

$$\ell(\mathbf{x}, y) = \mathrm{CE}\big(f(\mathbf{x}) + \tau \log \boldsymbol{\pi}_k,\ y\big), \qquad \tau = 1, \tag{15}$$

where $\boldsymbol{\pi}_k$ is floored at 10^{-4}. The gradient for an absent class becomes $q'_c \propto \pi_{k,c}^{\tau} e^{f_c} \approx 10^{-4} e^{f_c}$, so absent classes are no longer pushed down, and f learns class-balanced scores. Second, the rows of the shared classifier are aggregated class-wise [14], [15],

$$[W]_c^{t+1} = \frac{\sum_{k \in \mathcal{S}_t} n_{k,c} [W_k]_c}{\sum_{k \in \mathcal{S}_t} n_{k,c}} \quad \text{(the same for } b_c\text{)}, \tag{16}$$

and a row that no participant observes keeps its value. Third, decisions are calibrated with the global prior (Eq. (17)). Full-model methods (FedAvg and V0) also use a larger learning rate for the heads than for the body (discriminative learning rates), without which the rows of rare classes barely move within the training budget.

H. Inference

1) Enrolled users: FedPEFT-IMU predicts $\hat{y} = \arg\max_c [h_k(\mathbf{e})]_c$ with the user's rank, local parts and personal head. A shared classifier trained with Eq. (15) outputs class-balanced scores, which we convert into decisions with a tunable prior strength,

$$\hat{y} = \arg\max_c \ f_c(\mathbf{x}) + \tau_{\mathrm{inf}} \log \pi_c. \tag{17}$$

$\tau_{\mathrm{inf}} = 0$ gives balanced decisions and $\tau_{\mathrm{inf}} = 1$ the accuracy-optimal decisions under $\boldsymbol{\pi}$; macro-F1 is best in between. We select $\tau_{\mathrm{inf}} = 0.5$ from $\{0, 0.25, 0.5, 0.75, 1\}$ on the validation users and keep it fixed for all methods and encoders.

2) Federated temporal decoding: Activities last minutes to hours, whereas the classifier sees a single window. Following the Capture-24 benchmark [16], [17], we optionally decode each user's chronological sequence with a hidden Markov model (HMM) whose hidden states are the activities. The transition matrix is estimated from the aggregated counts with add-one smoothing, $\Gamma_{ij} = (1 + \sum_k N_{k,ij}) / \sum_{j'} (1 + \sum_k N_{k,ij'})$, so it is learned federatedly from C^2 integers per client without sharing any label sequence. The emission score of window $\mathbf{x}$ for state c is the class-balanced score,

$$\log p(\mathbf{x} \mid c) \simeq \log \mathrm{softmax}\big(f(\mathbf{x}) - (1-\tau) \log \tilde{\boldsymbol{\pi}}\big)_c, \tag{18}$$

with $\tau = 1$ and $\tilde{\boldsymbol{\pi}} = \boldsymbol{\pi}$ for a logit-adjusted shared classifier. For a personal classifier ($\tau = 0$) we divide by the client's prior shrunk halfway towards the global one, $\tilde{\boldsymbol{\pi}} = \frac{1}{2}\boldsymbol{\pi}_k + \frac{1}{2}\boldsymbol{\pi}$; dividing by $\boldsymbol{\pi}_k$ alone would boost classes that the client has never observed by up to $-\log 10^{-4} \approx 9$ nats. The Viterbi algorithm [18] returns the most likely activity sequence in $\mathcal{O}(|\mathcal{D}_k^{\mathrm{te}}|\, C^2)$ time. Sequences are split wherever two consecutive windows are more than three nominal spacings apart, so the decoder never smooths across recording gaps. Decoding runs on the device; it uses only the shared Γ and $\boldsymbol{\pi}$.

3) New users: A user u who never participated downloads the global state (rank $r_{\max}$) and the prototypes. *Zero-shot*: encoders with BatchNorm first re-estimate their normalization statistics on the user's *unlabelled* windows (AdaBN [19]), since the global model has no population-level statistics after training with local norms; the user is then served by the nearest-prototype rule

$$\hat{y} = \arg\max_{c \in \mathcal{C}_P} \ \cos\big(\mathbf{e}, \mathbf{P}_c\big). \tag{19}$$

Few-shot: with m minutes of labels per activity, a new personal head is initialized at the zero-shot solution, $[W_u]_c = \mathbf{P}_c / \|\mathbf{P}_c\|$ and $b_{u,c} = 0$, since $\arg\max_c \cos(\mathbf{e}, \mathbf{P}_c) = \arg\max_c \mathbf{P}_c^{\top}\mathbf{e} / \|\mathbf{P}_c\|$. The head, the norms and the input adapter $\{h_u, \psi_u, a_u\}$ are then fine-tuned on the shots. Because the shots are roughly class-balanced, unlike a real day, this fit uses Eq. (15) with the prior of the shots, and decisions use Eq. (17).

I. Training Procedure

Algorithm 1 summarizes training. Each device is assigned a rank tier once, and each round consists of download, local training of the trainable roles, prototype computation, upload of the federated roles and statistics, and server aggregation. Because only a fraction ρ of the clients participates in a round, the personal parts of a client may lag behind the latest global representation. After the last round, every client therefore performs a short personalization pass (8 steps) that updates only ϕ_k against the final global model, without any communication.

J. Communication, Memory and Computation

1) Uplink: Per round, client k uploads

$$U_k = \underbrace{r_k \textstyle\sum_{\ell \in \mathcal{L}} \big(d_\ell^{\mathrm{in}} + d_\ell^{\mathrm{out}}\big)}_{\text{LoRA}} + \underbrace{|\theta|}_{\text{head}} + \underbrace{C(D+1)}_{\text{prototypes, counts}} \tag{20}$$

values, which is linear in the rank and independent of the size of the frozen encoder. Full fine-tuning uploads $|\Omega_0| + |\theta| + |h|$ values. Table III lists the resulting budgets for the four encoders together with the measured uplink, which ranges from $0.03\,\mathrm{MB}$ to $1.2\,\mathrm{MB}$ per update for FedPEFT-IMU versus 0.23–$42\,\mathrm{MB}$ for full fine-tuning.

Algorithm 1: FedPEFT-IMU training

Input : clients $k = 1..K$ with $\mathcal{D}_k^{\mathrm{tr}}$; tiers $\mathcal{R}$; rounds T; rate ρ; local steps S; weight λ; pretrained Ω_0
Output : global state $(\{\bar{W}_\ell, B_\ell, A_\ell\}_\ell, \theta, \{\mathbf{P}_c\}, \boldsymbol{\pi}, \Gamma)$; local parts $\{\phi_k\}$

Server: $\bar{W}_\ell \leftarrow \Omega_{0,\ell}$, $B_\ell \leftarrow 0$, $A_\ell \leftarrow \mathrm{Kaiming}(r_{\max})$, $\theta \leftarrow$ init, $\mathcal{C}_P \leftarrow \emptyset$;
for *each client* k **do**
  $r_k \sim \mathrm{Uniform}(\mathcal{R})$;
  $\phi_k \leftarrow$ (identity a_k, pretrained ψ_k, random h_k);
  send $\{n_{k,c}\}_c$ and N_k to the server ; *// once*
Server: build the prior $\boldsymbol{\pi}$ and the HMM transitions Γ;
for $t = 0, \ldots, T-1$ **do**
  $\mathcal{S}_t \leftarrow$ random subset of $\lceil \rho K \rfloor$ clients;
  for $k \in \mathcal{S}_t$ **in parallel do**
    receive $\{\bar{W}_\ell, B_{\ell,[r_k]}, A_{\ell,[r_k]}\}_\ell$, θ, $\{\mathbf{P}_c\}_{c \in \mathcal{C}_P}$;
    S AdamW steps on F_k (Eq. (5)) w.r.t. $\{A_\ell, B_\ell\}, \theta, \phi_k$;
    compute $\mathbf{p}_{k,c}$ by Eq. (3); keep ϕ_k on the device;
    upload $\{A_{k,\ell}, B_{k,\ell}\}_\ell$, θ_k, $\{\mathbf{p}_{k,c}, n_{k,c}\}_c$, n_k;
  Server: for every $\ell \in \mathcal{L}$ apply Eqs. (8) to (11);
  Server: $\theta \leftarrow \sum_k \tilde{p}_k \theta_k$; prototypes by Eq. (14);
for *each client* k **do** *// final personalization, no communication*
  8 steps on F_k w.r.t. ϕ_k only, at the final global state;

2) Downlink: In addition to the truncated factors, the head and the prototypes, exact aggregation changes the frozen weights $\bar{W}_\ell$, which a client must receive before it trains again. In our accounting we conservatively charge the full dense $\bar{W}_\ell$ of every adapted layer at each download. This term can be reduced substantially: if the truncated SVD in Eq. (10) is exact, the per-round residual in Eq. (11) has rank at most $\sum_{k \in \mathcal{S}_t} r_k$, because $\mathrm{rank}(M) \le r_{\max} + \sum_{k \in \mathcal{S}_t} r_k$, and it can therefore be transmitted in factored form. We report the conservative figure and return to this point in the discussion.

3) Client memory and computation: Only LoRA factors, the heads and ϕ_k require gradients and optimizer states, i.e., three additional floating-point values per trainable parameter for AdamW [20]. Activation memory still scales with the encoder, so the memory saving is largest when the optimizer state dominates (large convolutional encoders) and modest for small Transformers. Prototype computation needs one extra forward pass over at most 512 windows per round.

4) Server computation: Forming M_ℓ costs $\mathcal{O}\big(d_\ell^{\mathrm{out}} d_\ell^{\mathrm{in}} \sum_{k \in \mathcal{S}_t} r_k\big)$, and the randomized SVD costs $\mathcal{O}\big(d_\ell^{\mathrm{out}} d_\ell^{\mathrm{in}} (r_{\max} + 8)\big)$ per layer and round. The server streams the uploads into running sums, so its memory does not grow with the number of participating clients.

K. Privacy Considerations

Raw windows, labels, label sequences, local normalization statistics, input adapters and personal classifiers never leave the device. The uploads are the LoRA factors and the projection head (as in any PEFT-based federated method), C class-mean embeddings in a 64-dimensional space and their counts, and, once, $C + C^2$ label statistics. Prototypes and counts reveal which activities a user performed, so they are as sensitive as a coarse activity diary; they can be protected by adding calibrated Gaussian noise to $\mathbf{p}_{k,c}$ and $n_{k,c}$ or by aggregating them with secure aggregation [25], because Eq. (14) is a ratio of two sums. The products $B_k A_k$ in Eq. (9) are also sums over clients, but secure aggregation of their dense form would forgo the uplink savings. A formal privacy analysis is beyond the scope of this work.

IV. Dataset and Experimental Protocol

A. The Capture-24 Dataset

We evaluate on Capture-24 [16], one of the largest publicly available free-living datasets of wrist-worn accelerometry with activity annotations. Its free-living protocol, its size and its natural division into individuals make it a realistic testbed for federated HAR. Participants are naturally treated as clients, so no synthetic partition is needed. The data originate from the CAPTURE-24 study (mainly 2014–2015), which compared time-use diaries with wearable sensors [26]. Members of the public in Oxfordshire, United Kingdom, wore an Axivity AX3 tri-axial accelerometer on the dominant wrist for about $24\,\mathrm{h}$, recording at $100\,\mathrm{Hz}$ with a dynamic range of $\pm 8\,g$. Ground truth during waking hours came from a wearable camera worn on a lanyard (OMG Life Autographer), which took a photograph every 20–40 s, and during the night from a sleep diary. Trained annotators, who had to reach an inter-rater agreement of Cohen's $\kappa > 0.8$ on a gold-standard set, labelled the recordings with codes from the Compendium of Physical Activities [27]. The camera protocol followed an established ethical framework [28]. The images are not released, and for de-identification the ages are binned and the dates randomized. Of 166 recruited participants, 151 remained after quality control, providing $3{,}883\,\mathrm{h}$ of recordings of which $2{,}562\,\mathrm{h}$ are annotated with 206 distinct codes. The dataset is distributed under the CC BY 4.0 licence.

The 206 codes are mapped to the six-class scheme of Willetts *et al.* [29], a standard benchmark task on this dataset: *sleep*, *sit/stand*, *mixed activity*, *walking*, *vehicle* and *bicycling* (Table IV). The task is strongly imbalanced. Sleep and sitting/standing account for $36.9\,\%$ and $39.0\,\%$ of the data, whereas bicycling accounts for only $1.0\,\%$ (an imbalance ratio of $40{:}1$), and only 42 of the 151 participants cycle at all. The participants are 99 women and 52 men in four age groups (Table V).

B. Preprocessing

Each participant's recording is processed independently, as it would be on the device. The released signals already include the authors' sampling-rate correction and gravity autocalibration [16]. We (i) split the stream into continuous segments wherever two consecutive timestamps are more than 1.5 sample periods apart; (ii) resample each segment from 100 to $30\,\mathrm{Hz}$ with an anti-aliased polyphase filter, the rate at which ssl-wearables and HALO operate and from which the $20\,\mathrm{Hz}$ encoders are derived by Π_E; (iii) cut non-overlapping $10\,\mathrm{s}$ windows (3×300 samples, in g); and (iv) label each window with the majority label of its $1{,}000$ raw samples, discarding windows whose majority is unannotated. This yields $921{,}656$ labelled windows ($2{,}560\,\mathrm{h}$). This total agrees with the $2{,}562$ annotated hours of the release and the $922{,}199$ windows of the official benchmark [16], and the small difference comes from the majority rule. Per participant, the labelled time ranges

TABLE III
PARAMETER BUDGET PER ENCODER AND MEASURED COMMUNICATION PER CLIENT UPDATE (FP32 SERIALIZATION, NO COMPRESSION). $|\Omega_0|$: FROZEN ENCODER WEIGHTS; LORA: TRAINABLE ADAPTER PARAMETERS AT RANK $r = 2/4/8$; $|\theta|$: SHARED HEAD; $|\phi_k|$: LOCAL PARAMETERS (NORM AFFINE, ADAPTER, CLASSIFIER).

Encoder	LoRA targets	$\lvert\Omega_0\rvert$	LoRA (r=2/4/8)	$\lvert\theta\rvert$	$\lvert\phi_k\rvert$	Uplink FedPEFT-IMU r=2/4/8 (MB)	Uplink full FT (MB)	Ratio†
LIMU-BERT [21]	W_q, W_v (shared layers)	51.1 k	0.6/1.2/2.3 k	4.8 k	0.8 k	0.023/0.025/0.030	0.227	8.6×
ssl-wearables [22]	all Conv1d kernels	10.4 M	60/119/239 k	67.6 k	12.1 k	0.511/0.750/1.227	42.15	49.8×
UniMTS [23]	graph + temporal convs	5.17 M	46/91/182 k	33.9 k	9.0 k	0.319/0.502/0.866	20.89	36.4×
HALO [24]‡	W_q, W_v (all blocks)	4.60 M	14/28/55 k	17.0 k	9.6 k	0.125/0.180/0.291	18.50	91.3×

†Mean uplink of full fine-tuning divided by the mean uplink of FedPEFT-IMU over all client updates (tiers drawn uniformly). ‡Our re-implementation of the HALO encoder, pretrained self-supervised on the pool users (Section IV-E).

TABLE IV
CAPTURE-24 CLASSES (WILLETTS 2018 SCHEME) AFTER WINDOWING, OVER ALL 151 PARTICIPANTS. CODES: DISTINCT COMPENDIUM ANNOTATIONS MAPPED TO THE CLASS. USERS: PARTICIPANTS WITH AT LEAST ONE WINDOW OF THE CLASS. CLIENTS: SHARE OF THE 106 FL CLIENTS THAT HAVE THE CLASS IN THEIR TRAINING PART. BOUT: MEDIAN LENGTH OF UNINTERRUPTED BOUTS.

Class	Content (sub-labels)	Codes	Windows	Share (%)	Users	Clients (%)	Bout (min)
Sleep	Sleep (sleep diary)	1	340,081	36.9	151	98	189.5
Sit/stand	Sitting, standing	97	359,521	39.0	151	100	8.2
Mixed activity	Household chores, manual work, sports, other	84	118,319	12.8	151	100	5.0
Walking	Walking	18	60,040	6.5	151	100	2.5
Vehicle	Travelling in a vehicle	5	34,736	3.8	91	58	11.8
Bicycling	Cycling	1	8,959	1.0	42	26	8.3
Total		206	921,656	100.0	151	–	–

from 6.1 to 24.7 h (median 17.1 h). Windows are stored in half precision with their start time, which the temporal decoder needs.

To keep the full experimental grid (four encoders, nine training configurations, centralized and federated) tractable on a single GPU, the experiments use one window per minute, i.e., every sixth window of each participant. This sub-sampling preserves each participant's diurnal sequence, label proportions and bout structure at a 1-min resolution. It yields 153,672 windows, of which the federated clients hold 85,878 training and 21,518 test windows. The pipeline supports the full resolution without modification.

C. Federated Partition and Data Splits

The split is made in two steps, first by user and then by time (Table V).

User level. The 151 participants are randomly assigned, with a fixed seed, to three disjoint groups: 106 *FL clients* (70 %), which take part in federated training; 30 *held-out users* (20 %), who are never seen during training and measure generalization to new users; and 15 *pool users* (10 %). The pool users provide *unlabelled* data for the self-supervised pretraining of HALO and serve as validation users for early stopping and for the choice of τ_{inf}. No test data are used for any model or hyperparameter selection. Each client corresponds to exactly one participant, so label skew, quantity skew and feature shift are all natural.

Time level. Within each FL client, the first 80 % of the chronologically ordered windows form the training part and the last 20 % the test part. A random window-level split would place temporally adjacent and nearly identical windows on both sides and inflate the scores; the chronological split instead asks the model to recognize the user's *future* activities. For each held-out user, the first 50 % is the adaptation part, from which few-shot labels are drawn, and the last 50 % the test part on which every new-user protocol is scored.

TABLE V
USER-LEVEL PARTITION (SEED 0), EXPERIMENT WINDOWS (ONE PER MINUTE) AND DEMOGRAPHICS. F/M: FEMALE/MALE. AGE GROUPS: 18–29 / 30–37 / 38–52 / ≥53 YEARS. USE: CHRONOLOGICAL SPLIT OF EACH USER'S WINDOWS (PERCENT), OR ROLE OF THE POOL USERS.

Group	Users	Windows	F/M	Age groups	Use
FL clients	106	107,396	66/40	31/25/26/24	80/20 train/test
Held-out	30	30,460	20/10	9/6/10/5	50/50 adapt/test
Pool	15	15,816	13/2	3/6/1/5	SSL, validation

The centralized references are trained on the union of the clients' training parts and tested on the same per-client test parts, so centralized and federated results are directly comparable. The user partition, the device-tier assignment and the client sampling are seeded. The partition files are released with the code.

D. Statistical Heterogeneity of the Natural Partition

Federated methods are commonly evaluated on synthetic partitions in which client label distributions are drawn from a Dirichlet distribution [30]. The natural partition of Capture-24 differs from these partitions in ways that matter for the design of Section III. Figure 3 and Table VI quantify them.

Skew is concentrated in the rare classes. All clients observe sit/stand, mixed activity and walking, and nearly all observe sleep, but only 58 % observe vehicle and 26 % bicycling.

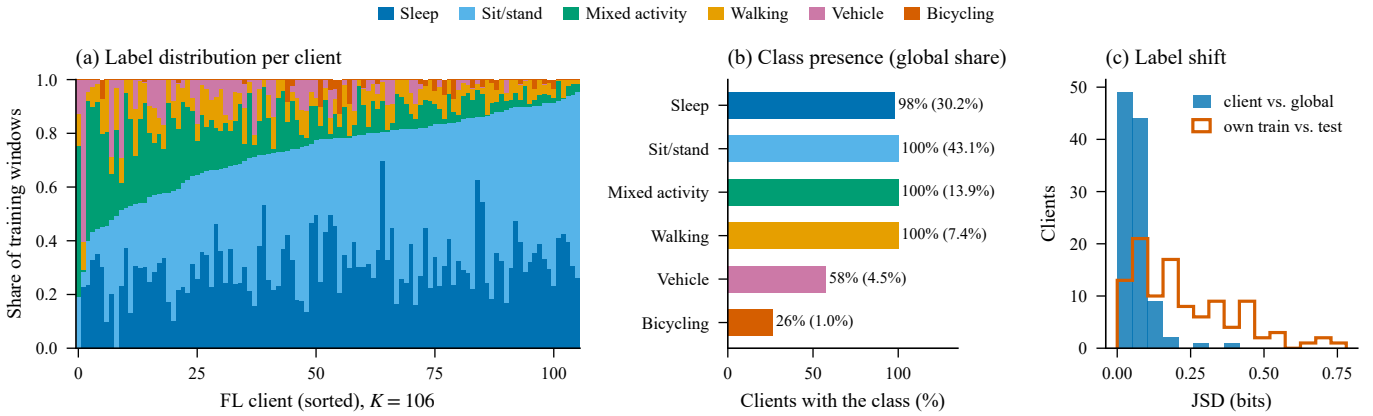


Fig. 3. Natural heterogeneity of the 106 Capture-24 clients (training parts, one window per minute). (a) Label distribution of every client. (b) Share of clients that observe each class (global share in parentheses); bicycling and vehicle are absent from most clients. (c) Jensen–Shannon divergence of each client's label distribution from the global one, and between the same client's training and test parts. The temporal shift within a client exceeds the shift between clients.

TABLE VI
STATISTICAL HETEROGENEITY OF THE FL CLIENTS (TRAINING PARTS UNLESS STATED). JSD: JENSEN–SHANNON DIVERGENCE IN BITS. EQUIVALENT α: CONCENTRATION OF A $\mathrm{Dir}(\alpha C\boldsymbol{\pi})$ LABEL-SKEW MODEL WITH THE SAME CLIENT SIZES AND THE SAME MEAN JSD.

Statistic	Value
Windows per client (min / median / max)	302 / 816 / 1156
Quantity skew: coefficient of variation; Gini	0.17; 0.09
Classes per client, mean (histogram)	4.82 (4: 27, 5: 71, 6: 8)
Clients observing all 6 classes	7.5 %
Share of the dominant class per client, mean	48.9 %
JSD(client ‖ global), mean ± sd (max)	0.065 ± 0.053 (0.387)
Equivalent Dirichlet concentration α	1.79
JSD(train ‖ test) within a client, mean (max)	0.226 (0.765)
Clients whose test part has a class absent from their train part	2.8 %
Test windows of such locally unseen classes	1.2 %

A client observes 4.82 of the six classes on average, and only 8 % observe all of them. The mean divergence from the global distribution (0.065 bits) corresponds to a Dirichlet concentration of about 1.79, a moderate skew by the standards of synthetic benchmarks. Its structure, however, is different: the rare classes are not under-represented but *absent* on most clients. This is the regime in which a shared classifier trained with cross-entropy loses rare classes (Section III-G).

Temporal shift exceeds client shift. Because each recording covers roughly one day, a chronological split separates different parts of the day. On average, sleep accounts for 30 % of a client's training part but 60 % of its test part, and the test part is mostly sleep for 57 % of the clients. The divergence between a client's own training and test parts (mean 0.226 bits) is more than three times larger than the divergence between clients. Decision rules that copy the local training prior are therefore miscalibrated even for enrolled users, which motivates calibrating decisions with the population prior (Eq. (17)) and modelling temporal context explicitly. In 3 % of the clients, the test part contains a class that the client never observed in training (1.2 % of all test windows), a case that only shared knowledge can handle.

Quantity skew is mild. The fixed 24-h protocol makes client sizes similar (302–1156 training windows, Gini coefficient 0.09). The weights $\tilde{p}_k$ are therefore close to uniform, and the heterogeneity is predominantly in the labels and the users' movement patterns.

Activities are persistent. At the 1-min resolution of the experiments, the diagonal of the aggregated transition matrix ranges from 0.77 (walking) to 1.00 (sleep), and the median bout lasts from 2.5 min (walking) to more than 3 h (sleep; Table IV). Window-level classifiers ignore this structure; the federated HMM of Section III-H exploits it at the cost of 42 integers per client.

E. Pretrained Encoders

To test whether the conclusions depend on the backbone, we instantiate FedPEFT-IMU with four IMU encoders that differ in architecture (Transformer, residual CNN, spatio-temporal graph network, patch-channel Transformer), in size (51 k to 10.5 M frozen parameters), in pretraining corpus and objective, and in input format (Table VII). Three are used with their official checkpoints. The weights are loaded into re-implementations with identical parameter names, and loading fails if less than 90 % of the tensors match. HALO [24] has not released weights. We therefore re-implemented its encoder at a reduced width ($d = 192$) and pretrained it with its stage-1 objectives (masked reconstruction of 50 % of the 1-s patches [31] and a patch-level InfoNCE loss [32] at temperature 0.2) on the 94,847 unlabelled windows of the pool users. Two documented deviations apply: learned channel embeddings replace the text-derived sensor descriptions, and each token receives the patch mean and log standard deviation, which per-patch z-scoring would otherwise remove. Because the pool users are disjoint from the clients and the held-out users, this pretraining leaks no evaluation data. Capture-24 provides only an accelerometer, so the input adapter feeds zeros to LIMU-BERT's gyroscope channels at initialization, and UniMTS receives the signal on the right-wrist joint of its 22-joint skeleton with zeros elsewhere, as prescribed by its authors [23].

TABLE VII
PRETRAINED IMU ENCODERS. INPUT: CHANNELS AFTER THE INPUT ADAPTER, SAMPLING RATE, SAMPLES PER WINDOW (DURATION) AND UNITS, PRODUCED FROM THE CANONICAL 3×300 WINDOW BY Π_E.

Encoder	Architecture	Pretraining data and objective	Input	d_z	$\lvert\Omega_0\rvert$
LIMU-BERT [21]	Transformer, 4 weight-shared layers (ALBERT-style), $d = 72$, 4 heads, mean pooling	HHAR smartphones and smartwatches [33]; masked-span reconstruction (official checkpoint)	6 (acc+gyro), 20 Hz, 120 (6 s), g	72	51 k
ssl-wearables [22]	1-D ResNet-V2 (harnet10), 5 stages, 64–1024 channels, anti-aliased down-sampling	UK Biobank, 700 k person-days of wrist Axivity AX3 data; multi-task self-supervision (arrow of time, permutation, time warping)	3, 30 Hz, 300 (10 s), g	1024	10.4 M
UniMTS [23]	ST-GCN [34], 10 blocks over a 22-joint skeleton graph, 64–512 channels	IMU signals synthesized from motion capture (HumanML3D [35]); contrastive alignment with LLM-augmented text descriptions	3 on wrist joint, 20 Hz, 200 (10 s), $\mathrm{m\,s^{-2}}$	512	5.2 M
HALO [24] (re-impl.)	1-s patch $\times$ channel tokens (temporal, spectral and statistics branches), 8 alternating temporal/cross-channel attention layers, $d = 192$, 6 heads, query pooling	Capture-24 pool users (unlabelled, disjoint from all evaluation users); masked reconstruction + patch contrastive, 8 epochs	3, 30 Hz, 300 (10 s), g	256	4.6 M

F. Compared Methods

Table VIII lists the federated methods, each defined by the roles of Table II. FedAvg trains the whole model from a random initialization. V0 and V1 are the upper-cost and lower-cost ways of using a pretrained encoder (full fine-tuning and a frozen encoder with trainable heads). V2 and V3 isolate the aggregation rule for homogeneous LoRA: factor averaging versus exact aggregation [6]. V4 adds rank heterogeneity with the delta-form aggregation of Section III-E, and it is the ablation of FedPEFT-IMU (V5) without local parts and prototypes. All methods with a shared classifier use the label-skew remedies of Section III-G, and all methods are scored with the same decoders. Two centralized references are trained on the pooled training parts: *from scratch*, which defines the target F1, and *pretrained fine-tuning*, an upper reference without privacy constraints.

G. Evaluation Metrics

Recognition. The primary metric is the *macro-F1* over the six classes, computed on the pooled test windows of all clients, so that rare activities weigh as much as frequent ones. We also report accuracy.

Fairness. For every client we compute its macro-F1 over the classes present in its own test part (a user who never cycles is not scored on cycling). We report the *worst-10 % client F1*, i.e., the mean over the $\lceil 0.1K \rceil = 11$ lowest-scoring clients, and the standard deviation of the client F1 values.

Communication. Every message is an explicit set of tensors. Its size is measured as the serialized size of uncompressed arrays (four bytes per real value, eight per integer), as in Flower's NumPy serialization [36], and includes every tensor that is sent: factors, heads, residual weights, prototypes and counts. The one-shot statistics are charged in the first round. We report the *uplink per client update* (MB), the *rounds to target*, i.e., the first evaluated round whose macro-F1 reaches $90\,\%$ of the centralized from-scratch macro-F1 of the same encoder (evaluations every 8 rounds), and the cumulative uplink of all clients up to that round (*MB to target*). No compression or quantization is applied, since it would benefit all methods equally.

Client cost. The *client memory* is the size of all model tensors plus the peak additional device memory allocated during a local update (activations, gradients and optimizer states), measured with the CUDA allocator. The *client time* is the wall-clock time of one local update.

New users. On the 30 held-out users we report the mean per-user macro-F1 on their test halves: *zero-shot* (no labels) and *few-shot* with $m \in \{1, 5\}$ minutes of labels per activity, drawn at random from the adaptation half. For methods with a shared classifier, the few-shot protocol re-fits the classifier on the frozen embeddings.

Decoding. Every metric is reported per window and after federated HMM decoding ("+HMM"). For methods with a shared classifier, we also report the common "FedAvg + local fine-tuning" protocol of personalized FL ("+FT"): every client fine-tunes only the classifier on its own training part (8 steps, the same loss as in training) and is scored with this personal classifier. FedPEFT-IMU needs no such step, because its classifier is already personal.

H. Implementation Details

All methods run in one transparent federated simulator written in PyTorch 2.11 (Python 3.13). All windows reside in GPU memory, so a mini-batch is an index tensor and every method sees exactly the same data stream for a given seed. Experiments ran on a single NVIDIA GeForce GTX 1660 Ti (6 GB) in fp32. Table IX lists the hyperparameters. They are identical for all encoders except UniMTS. Its graph network costs about 7 GMAC per window, so it trains with a reduced budget (40 rounds, batch size 16, 20 few-shot steps, intermediate evaluations on 60 test windows per client and at most 200 test windows per held-out user). UniMTS results are therefore comparable across methods, but not with the other

TABLE VIII
FEDERATED METHODS. Ω: ENCODER WEIGHTS; g, h: SHARED AND CLASSIFIER HEADS; ψ: NORMALIZATION; a: INPUT ADAPTER; P: PROTOTYPES.

Method	Init.	Trained	Uploaded	Kept local	LoRA rank	Aggregation
FedAvg [8]	random	Ω, ψ, a, g, h	Ω, ψ, a, g, h	–	–	FedAvg
V0 Full fine-tuning	pretrained	Ω, ψ, a, g, h	Ω, ψ, a, g, h	–	–	FedAvg
V1 Head only	pretrained	g, h	g, h	–	–	FedAvg
V2 LoRA, factor averaging	pretrained	LoRA, g, h	LoRA, g, h	–	8	avg. A, avg. B
V3 LoRA, exact	pretrained	LoRA, g, h	LoRA, g, h	–	8	FedEx-LoRA [6]
V4 Heterogeneous ranks	pretrained	LoRA, g, h	LoRA, g, h	–	2/4/8	delta form + SVD (Section III-E)
V5 FedPEFT-IMU	pretrained	LoRA, g, h, ψ, a	LoRA, g, P	h, ψ, a	2/4/8	delta form + SVD, prototypes

TABLE IX
HYPERPARAMETERS (UNIMTS BUDGET IN PARENTHESES WHERE IT DIFFERS).

Federated training	
Rounds T; clients per round	80 (40); 21 of 106 ($\rho = 0.2$)
Local work per round	1 epoch, capped at $S = 8$ steps of batch 64 (16)
Learning rates	FedAvg 10^{-3}; V0 body 3×10^{-4}; LoRA, heads, ψ, a: 2×10^{-3}
Weight decay	10^{-4}
LoRA	$r = 8$ (V2, V3); tiers $\{2, 4, 8\}$ uniform (V4, V5); $\alpha = 2r$; dropout 0
Embedding D; prototype weight λ	64; 0.5 (prototypes on ≤ 512 windows)
Label skew (shared classifiers)	$\tau = 1$ with client prior; class-wise rows; prior floor 10^{-4}
Decisions	$\tau_{\text{inf}} = 0.5$ (shared), arg max (personal), nearest prototype (zero-shot)
HMM	add-one smoothing; split at gaps > 3 min
Final personalization; +FT	8 steps on ϕ_k; 8 steps on h
New users	
Few-shot labels; steps	1 and 5 min per activity; 60 (20)
FedPEFT-IMU few-shot	adapt h_u, ψ_u, a_u, batch 64 (16), lr 2×10^{-3}
Shared-classifier few-shot	re-fit h on frozen embeddings, Adam, lr 5×10^{-3}
Centralized references	
Epochs; early stopping	10 (5), cosine schedule; patience 3 (2) on validation macro-F1
Batch; learning rate	128 (16); scratch 10^{-3}, fine-tuning 3×10^{-4}
HALO pretraining (pool users)	
Schedule	8 epochs, batch 128, AdamW lr 3×10^{-4} (cosine), wd 10^{-5}
Objectives	mask ratio 0.5; InfoNCE temperature 0.2; scaling, jitter, time warping

encoders. All local updates use AdamW [20] with a fresh optimizer state in every round.

V. RESULTS

[Results: to be written.]

VI. CONCLUSION

[Conclusion: to be written.]

REFERENCES

[1] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[2] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 2089–2099.

[3] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," arXiv:1912.00818, 2019.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[5] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8432–8440.

[6] R. Singhal, K. Ponkshe, and P. Vepakomma, "FedEx-LoRA: Exact aggregation for federated and efficient fine-tuning of large language models," in *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[7] N. Halko, P.-G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[8] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 54, 2017, pp. 1273–1282.

[9] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[10] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.

[11] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[12] J. Ren, C. Yu, S. Sheng, X. Ma, H. Zhao, S. Yi, and H. Li, "Balanced meta-softmax for long-tailed visual recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 4175–4186.

[13] J. Zhang, Z. Li, B. Li, J. Xu, S. Wu, S. Ding, and C. Wu, "Federated learning with label distribution skew via logits calibration," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022, pp. 26 311–26 329.

[14] X.-C. Li and D.-C. Zhan, "FedRS: Federated learning with restricted softmax for label distribution non-IID data," in *Proc. 27th ACM SIGKDD Conf. Knowledge Discovery & Data Mining*, 2021, pp. 995–1005.

[15] X. Shang, Y. Lu, G. Huang, and H. Wang, "Federated learning on heterogeneous and long-tailed data via classifier re-training with federated features," in *Proc. 31st Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2022, pp. 2218–2224.

[16] S. Chan, H. Yuan, C. Tong, A. Acquah, A. Schonfeldt, J. Gershuny, and A. Doherty, "CAPTURE-24: A large dataset of wrist-worn activity tracker data collected in the wild for human activity recognition," *Scientific Data*, vol. 11, p. 1135, 2024.

[17] R. Walmsley, S. Chan, K. Smith-Byrne, R. Ramakrishnan, M. Woodward, K. Rahimi, T. Dwyer, D. Bennett, and A. Doherty, "Reallocation of time between device-measured movement behaviours and risk of incident cardiovascular disease," *British Journal of Sports Medicine*, vol. 56, no. 18, pp. 1008–1017, 2022.

[18] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[19] Y. Li, N. Wang, J. Shi, X. Hou, and J. Liu, "Adaptive batch normalization for practical domain adaptation," *Pattern Recognition*, vol. 80, pp. 109–117, 2018.

[20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[21] H. Xu, P. Zhou, R. Tan, M. Li, and G. Shen, "LIMU-BERT: Unleashing the potential of unlabeled data for IMU sensing applications," in *Proc. 19th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, 2021, pp. 220–233.

[22] H. Yuan, S. Chan, A. P. Creagh, C. Tong, A. Acquah, D. A. Clifton, and A. Doherty, "Self-supervised learning for human activity recognition using 700,000 person-days of wearable data," *npj Digital Medicine*, vol. 7, p. 91, 2024.

[23] X. Zhang, D. Teng, R. R. Chowdhury, S. Li, D. Hong, R. K. Gupta, and J. Shang, "UniMTS: Unified pre-training for motion time series," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[24] Z. Ding, L. Zhang, and X. Ouyang, "HALO: A heterogeneity-aware language-aligned IMU foundation model for open-set human activity recognition," arXiv:2608.27233, 2026.

[25] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.

[26] J. Gershuny, T. Harms, A. Doherty, E. Thomas, K. Milton, P. Kelly, and C. Foster, "Testing self-report time-use diaries against objective instruments in real time," *Sociological Methodology*, vol. 50, no. 1, pp. 318–349, 2020.

[27] B. E. Ainsworth, W. L. Haskell, S. D. Herrmann, N. Meckes, D. R. Bassett, C. Tudor-Locke, J. L. Greer, J. Vezina, M. C. Whitt-Glover, and A. S. Leon, "2011 Compendium of Physical Activities: A second update of codes and MET values," *Medicine & Science in Sports & Exercise*, vol. 43, no. 8, pp. 1575–1581, 2011.

[28] P. Kelly, S. J. Marshall, H. Badland, J. Kerr, M. Oliver, A. R. Doherty, and C. Foster, "An ethical framework for automated, wearable cameras in health behavior research," *American Journal of Preventive Medicine*, vol. 44, no. 3, pp. 314–319, 2013.

[29] M. Willetts, S. Hollowell, L. Aslett, C. Holmes, and A. Doherty, "Statistical machine learning of sleep and physical activity phenotypes from sensor data in 96,220 UK Biobank participants," *Scientific Reports*, vol. 8, p. 7961, 2018.

[30] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.

[31] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 16 000–16 009.

[32] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," arXiv:1807.03748, 2018.

[33] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proc. 13th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, 2015, pp. 127–140.

[34] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[35] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng, "Generating diverse and natural 3D human motions from text," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5152–5161.

[36] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.